

Design and Implementation of Azure Intelligent Search Application using NLP

Project Report

Submitted by: YADUKRISHNA RANJITH

Reg No: 23BBCAMR456

Date: 29/1/2026

Name of the Institute:

Yenepoya (Deemed to be University)

Industry Project Title

Azure Intelligent Search Application for Internal Self-Service Portal

Name of the Company

Microsoft (Azure Cloud Platform)

Start Date:

29-01-2026

End Date:

17-03-2026

Total Effort (hrs.): 40

Project Environment & Tools Used

- Cloud Platform: **Microsoft Azure**
- Search Engine: **Azure Cognitive Search**
- NLP Service: **Azure OpenAI Service**
- Storage: **Azure Blob Storage**
- Hosting: **Azure App Service**
- Programming: Python, REST APIs
- Frontend: HTML, CSS, JavaScript
- Monitoring: Azure Monitor

- Version Control: Git, GitHub

Table of Contents

1. Objective and Scope
2. Problem Statement and Existing Approaches
3. Approach and Tools & Technologies
4. Workflow
5. Assumptions and Implementation
6. Data Collection and Processing Steps
7. Diagrams and Visualizations
8. Solution Design
9. Challenges and Opportunities
10. Reflections on the Project
11. Outcome and Conclusion
12. Enhancement Scope
13. Research Questions and Responses
14. References

1. Objective and Scope Objective:

To design and implement an AI-powered enterprise search solution using Azure services that enables employees to search internal documents using natural language queries.

Scope:

- Index internal HR, finance, and technical documents
- Enable semantic search using NLP
- Provide secure role-based access
- Deploy via internal self-service portal

2. Problem Statement and Existing Approaches

Problem Statement

Traditional keyword-based search systems fail to understand user intent. Employees often receive irrelevant results, leading to inefficiency and productivity loss.

The organization requires an intelligent NLP-based search system capable of understanding context and retrieving accurate results. **Existing Approaches**

- Manual document browsing
 - Basic SQL-based search
 - Keyword filtering without semantic understanding
- These approaches lack AI-driven contextual intelligence.

3. Approach and Tools & Technologies Approach:

- Document ingestion into Azure Blob Storage
 - Index creation using Azure Cognitive Search
 - NLP enrichment using Azure OpenAI
 - Semantic ranking and query interpretation
 - Secure deployment via Azure App Service
- Tools & Technologies:**
- Azure Cognitive Search (Indexing & Ranking)
 - Azure OpenAI (Query Understanding & NLP)
 - Azure Blob Storage (Document Repository)
 - Azure App Service (Portal Hosting)
 - Azure Active Directory (Authentication)
 - REST APIs (Integration Layer)

4. Workflow

1. Upload internal documents to Blob Storage
 2. Configure search index
 3. Apply AI enrichment pipeline
 4. User submits natural language query
 5. Semantic search retrieves ranked results
 6. Results displayed in self-service portal
-

5. Assumptions and Implementation Assumptions

- Azure services are properly configured
- Users have Azure AD credentials
- Documents are in supported formats (PDF, DOCX)
- Internet connectivity is stable

Implementation Steps

1. Create Azure Blob Storage container
2. Upload sample internal documents
3. Create Azure Cognitive Search service
4. Configure index and data source
5. Integrate Azure OpenAI for semantic search
6. Develop web portal interface
7. Deploy via Azure App Service
8. Test search functionality

6. Data Collection and Processing Steps Data Sources:

- HR policies
 - Finance guidelines
 - IT documentation
 - Project reports
- ### Processing Steps:
- Text extraction from documents
 - Metadata tagging
 - NLP-based enrichment
 - Index generation
 - Semantic ranking

Azure Monitor logs were analyzed to validate query performance and response accuracy.

7. Diagrams & Visualizations

Figure 1: Architecture Workflow

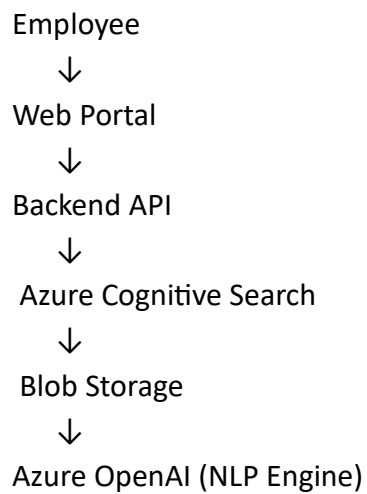
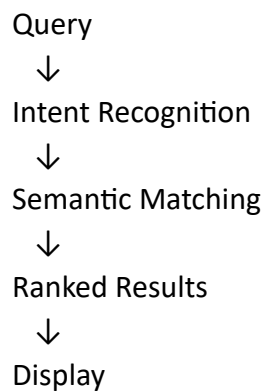


Figure 2: Search Process Flow



8. Solution Design

The solution follows a layered architecture:

- **Storage Layer:** Azure Blob Storage
- **Search Layer:** Azure Cognitive Search
- **AI Layer:** Azure OpenAI
- **Application Layer:** Web Portal
- **Security Layer:** Azure Active Directory
- **Monitoring Layer:** Azure Monitor

This modular design ensures scalability, reliability, and maintainability.

9. Challenges

- Configuring semantic ranking
- Handling large document datasets
- Ensuring secure authentication
- Optimizing response time

Opportunities

- Integration with chatbot
- Multilingual support
- AI-based summarization
- Enterprise-wide scalability

10. Reflections on the Project

This project provided hands-on experience in:

- Cloud architecture design
- Enterprise AI integration
- NLP-based semantic search
- Secure cloud deployment

It enhanced understanding of Azure cloud services and AI-powered enterprise applications.

11. Outcome and Conclusion

The project successfully implemented an intelligent search system using Azure services. The application:

- Reduced document search time
- Improved search accuracy
- Enabled natural language queries
- Enhanced employee productivity

This solution demonstrates the effectiveness of AI-driven enterprise search systems.

12. Enhancement Scope

- Conversational AI chatbot integration

- Voice-based search
- AI-generated document summaries
- Search analytics dashboard
- Multi-environment deployment (Dev, QA, Prod)

13. Research Questions and Responses

Q1: Why is semantic search important in enterprises?

A1: It improves search accuracy by understanding user intent rather than relying on keyword matching.

Q2: Why use Azure native services?

A2: They provide seamless integration, scalability, security, and enterprise-grade reliability.

Q3: How does NLP improve search performance?

A3: NLP enables context-aware query interpretation, improving relevance and ranking.

14. References

- Microsoft Azure Documentation: <https://learn.microsoft.com/azure>
- Azure Cognitive Search Docs
- Azure OpenAI Documentation
- Jurafsky & Martin (2019). *Speech and Language Processing*
- Enterprise Search Architecture Papers